

**LAPORAN TUGAS 1 SISTEM
OPERASI**



**Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom**

**Disusun Oleh:
Putri Suci Rahmadhani
(2410651008)**

**PRODI TEKNIK INFORMATIKA FAKULTAS
TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER 2025**

TUGAS 1

1. Implementasi

- Buat nama file
Nano process_manager.c
- Ketik codingan dalam file

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Ini child process, PID: %d\n", getpid());
    } else if (pid > 0) {
        printf("Ini parent process, PID: %d\n", getpid());
    } else {
        printf("Fork gagal!\n");
    }

    return 0;
}
```
- Perintah jalankan

```
gcc -o process_manager process_manager.c
./process_manager
```

2. Penjelasan

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

Baris ini memanggil tiga library penting:

- `stdio.h` → untuk fungsi input/output seperti `printf()`.
- `unistd.h` → untuk fungsi sistem seperti `fork()` dan `sleep()`.
- `sys/wait.h` → untuk fungsi `wait()` yang menunggu proses anak selesai.

```
int main() {
    pid_t pid1, pid2;
```

Deklarasi `main()` sebagai fungsi utama.
`pid_t` adalah tipe data khusus untuk menyimpan PID (Process ID).
`pid1` dan `pid2` digunakan untuk menyimpan hasil `fork()`.

```
// Membuat child process pertama
```

```
pid1 = fork();
```

Fungsi fork() digunakan untuk membuat proses baru (child).

Setelah baris ini dijalankan, akan ada dua proses:

- Parent: fork() mengembalikan nilai > 0 (PID anak)

- Child: fork() mengembalikan nilai 0

```
if (pid1 < 0) {  
    printf("Fork pertama gagal!\n");  
    return 1;  
}
```

Jika fork() gagal (misalnya memori tidak cukup), tampilkan pesan error dan keluar program.

```
if (pid1 == 0) {  
    // Proses anak pertama  
    printf("Child 1: PID = %d, Parent PID = %d\n", getpid(), getppid());  
    sleep(2); // simulasi kerja anak 1  
    printf("Child 1 selesai.\n");  
} else {
```

```
    // Hanya dijalankan oleh parent
```

```
    pid2 = fork(); // Membuat child kedua
```

Bagian ini hanya dijalankan oleh parent dari fork() pertama.

Parent membuat child kedua dengan fork() lagi.

```
if (pid2 < 0) {  
    printf("Fork kedua gagal!\n");  
    return 1;  
}
```

Jika fork kedua gagal, tampilkan pesan dan keluar.

```
if (pid2 == 0) {  
  
    // Proses anak kedua  
    printf("Child 2: PID = %d, Parent PID = %d\n", getpid(), getppid());  
    sleep(3); // simulasi kerja anak 2  
    printf("Child 2 selesai.\n");  
} else {
```

```
    // Parent menunggu kedua anak selesai
```

```
    wait(NULL);
```

```
    wait(NULL);
```

```
    printf("Parent: PID = %d - Semua child sudah selesai.\n", getpid());
```

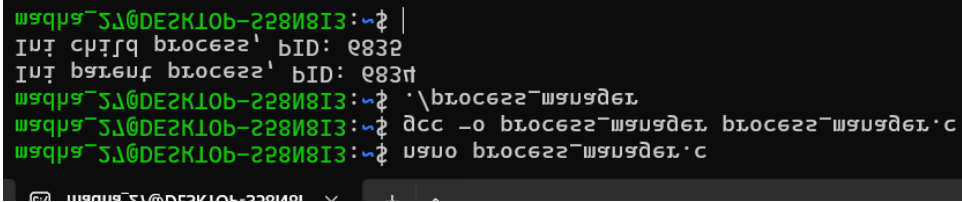
```
}
```

```
}
```

```
    return 0;  
}
```

Bagian akhir program menandakan selesai dengan sukses (return 0).

3. Pengujian dan hasil



```
msqmu@SADDE2K10b-228M8I3:~$ |  
INT cputq process2' bID: 0832  
INT bseuq process2' bID: 0832  
msqmu@SADDE2K10b-228M8I3:~$ .\process2-mpuadex  
msqmu@SADDE2K10b-228M8I3:~$ gcc -o process2-mpuadex process2.c  
msqmu@SADDE2K10b-228M8I3:~$ mpuadex process2-mpuadex.c
```

TUGAS 2

1. Implementasi

- Buat nama file
Nano multi_process.c
- Ketik codingan dalam file

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Ini child process, PID: %d\n", getpid());
    } else if (pid > 0) {
        printf("Ini parent process, PID: %d\n", getpid());
    } else {
        printf("Fork gagal!\n");
    }

    return 0;
}
```
- Perintah jalankan

```
gcc -o process_manager process_manager.c
./process_manager
```

2. Penjelasan

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

Baris ini berisi tiga library penting:

- `stdio.h` → untuk fungsi input/output seperti `printf()`.
- `unistd.h` → untuk fungsi sistem seperti `fork()`, `getpid()`, `getppid()`, dan `sleep()`.
- `sys/wait.h` → untuk fungsi `wait()` agar parent bisa menunggu child selesai.

```
int main() {
    pid_t pid1, pid2;
```

Fungsi utama program.

Tipe data `pid_t` digunakan untuk menyimpan ID proses (Process ID).

Variabel `pid1` dan `pid2` digunakan untuk menampung nilai hasil dari `fork()`.

```
pid1 = fork();
```

Fungsi fork() digunakan untuk membuat proses baru (anak).

Setelah baris ini dijalankan, program akan terbelah menjadi dua proses:

- Proses parent (induk) → fork() mengembalikan nilai positif (PID anak).
- Proses child (anak) → fork() mengembalikan nilai 0.

```
if (pid1 < 0) {  
    printf("Fork pertama gagal!\n");  
    return 1;  
}
```

Jika fork() gagal (misalnya karena memori tidak cukup), maka pid1 bernilai negatif dan program akan menampilkan pesan “Fork pertama gagal!” lalu keluar dari program.

```
if (pid1 == 0) {  
    printf("Child 1: PID = %d, Parent PID = %d\n", getpid(), getppid());  
    sleep(2);  
    printf("Child 1 selesai.\n");  
}
```

Jika pid1 == 0, berarti proses yang sedang berjalan adalah proses anak pertama.

getpid() digunakan untuk menampilkan ID proses anak.

getppid() menampilkan ID proses induknya.

sleep(2) membuat proses anak berhenti sejenak selama 2 detik (simulasi kerja).

Setelah itu muncul teks “Child 1 selesai.”

```
else {  
    pid2 = fork();
```

Jika pid1 bukan 0 (berarti parent), maka parent membuat proses anak kedua dengan fork() lagi dan hasilnya disimpan di pid2.

```
if (pid2 < 0) {  
    printf("Fork kedua gagal!\n");  
    return 1;  
}
```

Jika pembuatan child kedua gagal, maka program menampilkan “Fork kedua gagal!” dan berhenti.

```
if (pid2 == 0) {  
    printf("Child 2: PID = %d, Parent PID = %d\n", getpid(), getppid());  
    sleep(3);  
    printf("Child 2 selesai.\n");  
}
```

Jika pid2 == 0, maka bagian ini dijalankan oleh child kedua.

Child kedua menampilkan PID-nya dan PID parent-nya, lalu tidur selama 3 detik sebelum menampilkan “Child 2 selesai.”

```
else {  
    wait(NULL);
```

```
wait(NULL);  
printf("Parent: PID = %d - Semua child sudah selesai.\n", getpid());  
}
```

Bagian else terakhir hanya dijalankan oleh parent (induk).

Fungsi wait(NULL) membuat parent menunggu anaknya sampai selesai.

Karena ada dua anak, maka wait() dipanggil dua kali.

Setelah kedua child selesai, parent menampilkan "Parent: PID = ... - Semua child sudah selesai."

```
return 0;
```

```
}
```

Akhir program. Mengembalikan nilai 0 menandakan program berjalan dengan sukses.

3. Pengujian dan hasil

```
madha_27@DESKTOP-S58N8I3:~$ nano multi_process.c  
madha_27@DESKTOP-S58N8I3:~$ gcc -o multi_process multi_process.c  
madha_27@DESKTOP-S58N8I3:~$ ./multi_process  
Child 1: PID = 6848, Parent PID = 6847  
Child 2: PID = 6849, Parent PID = 6847  
Child 1 selesai.  
Child 2 selesai.  
Parent: PID = 6847 - Semua child sudah selesai.  
madha_27@DESKTOP-S58N8I3:~$ |
```

TUGAS 3

1. Implementasi

- Buat nama file
Nano custom_shell.c

- Ketik codingan dalam file
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```
#define MAX_CMD_LEN 100  
#define HISTORY_SIZE 5
```

```
int main() {  
    char command[MAX_CMD_LEN];  
    char history[HISTORY_SIZE][MAX_CMD_LEN];  
    int history_count = 0;  
  
    while (1) {  
        printf("MyShell> ");  
        fflush(stdout);  
  
        // Baca input dari user  
        if (fgets(command, sizeof(command), stdin) == NULL) {  
            printf("\n"); // jika Ctrl+D ditekan  
            break;  
        }  
  
        // Hapus newline '\n'  
        command[strcspn(command, "\n")] = '\0';  
  
        // Jika input kosong, lanjutkan ke prompt berikutnya  
        if (strlen(command) == 0) continue;  
  
        // Simpan ke history  
        strcpy(history[history_count % HISTORY_SIZE], command);  
        history_count++;  
  
        // Jika user mengetik "exit" -> keluar  
        if (strcmp(command, "exit") == 0) {  
            printf("Keluar dari MyShell.\n");  
            break;  
        }  
  
        // Jika user mengetik "history" -> tampilkan 5 perintah terakhir
```



```

        else if (strcmp(command, "history") == 0) {
            printf("=== 5 Perintah Terakhir ===\n");
            int start = history_count - HISTORY_SIZE;
            if (start < 0) start = 0;
            for (int i = start; i < history_count; i++) {
                printf("%d. %s\n", i + 1, history[i % HISTORY_SIZE]);
            }
            continue;
        }

        // Jalankan perintah sistem
        system(command);
    }

    return 0;
}

```

- Perintah jalankan
gcc -o custom_shell custom_shell.c
./custom_shell

2. Penjelasan

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Baris ini memanggil tiga library penting:

- `stdio.h` → untuk fungsi input/output seperti `printf()`, `fgets()`, dan `fflush()`.
- `stdlib.h` → untuk fungsi `system()` dan `exit()` yang digunakan menjalankan perintah sistem.
- `string.h` → untuk fungsi manipulasi string seperti `strcpy()`, `strcmp()`, `strlen()`, dan `strcspn()`.

```

#define MAX_CMD_LEN 100
#define HISTORY_SIZE 5

```

Baris ini mendefinisikan dua konstanta:

- `MAX_CMD_LEN = 100` → panjang maksimum perintah yang bisa diketik pengguna.
- `HISTORY_SIZE = 5` → jumlah maksimum perintah terakhir yang disimpan dalam riwayat (history).

```

int main() {
    char command[MAX_CMD_LEN];
    char history[HISTORY_SIZE][MAX_CMD_LEN];
    int history_count = 0;

```

Deklarasi variabel:

- command → menyimpan perintah yang diketik user.
- history → array dua dimensi untuk menyimpan daftar perintah sebelumnya.
- history_count → menghitung total berapa kali user mengetik perintah.

```
while (1) {  
    printf("MyShell> ");  
    fflush(stdout);
```

Perulangan while(1) digunakan agar shell terus berjalan sampai user mengetik "exit".

printf("MyShell> ") menampilkan prompt seperti di terminal.

fflush(stdout) memastikan teks prompt langsung muncul tanpa menunggu buffer.

```
if (fgets(command, sizeof(command), stdin) == NULL) {  
    printf("\n");  
    break;  
}
```

Bagian ini membaca input dari user menggunakan fgets().

Jika user menekan Ctrl+D (EOF), maka fgets() akan mengembalikan NULL, dan program akan keluar dari perulangan.

```
command[strcspn(command, "\n")] = '\0';
```

Fungsi strcspn() digunakan untuk mencari posisi karakter newline '\n' dalam string.

Kemudian newline tersebut diubah menjadi '\0' agar tidak ikut terbaca saat menjalankan perintah.

```
if (strlen(command) == 0) continue;
```

Jika user tidak mengetik apapun (input kosong), maka loop lanjut ke iterasi berikutnya tanpa melakukan apa-apa.

```
strcpy(history[history_count % HISTORY_SIZE], command);  
history_count++;
```

Dua baris ini menyimpan perintah user ke dalam array history.

Operator modulo (%) digunakan agar hanya 5 perintah terakhir yang disimpan (sirkuler).

Misal jika history_count sudah 6, maka perintah baru akan menimpa perintah paling lama.

```
if (strcmp(command, "exit") == 0) {  
    printf("Keluar dari MyShell.\n");  
    break;  
}
```

Jika user mengetik "exit", maka program menampilkan pesan keluar dan berhenti dari loop utama.

```
else if (strcmp(command, "history") == 0) {  
    printf("=== 5 Perintah Terakhir ===\n");  
    int start = history_count - HISTORY_SIZE;
```

```

    if (start < 0) start = 0;
    for (int i = start; i < history_count; i++) {
        printf("%d. %s\n", i + 1, history[i % HISTORY_SIZE]);
    }
    continue;
}

```

Jika user mengetik “history”, maka program menampilkan daftar maksimal 5 perintah terakhir yang pernah diketik.

- Variabel start menentukan dari perintah ke berapa daftar dimulai.
- Loop for mencetak semua isi history dengan format nomor urut dan teks perintah.
- continue digunakan agar setelah menampilkan history, program kembali ke prompt tanpa menjalankan system().

```
system(command);
```

Fungsi system() digunakan untuk menjalankan perintah yang diketik user di terminal shell Linux asli.

Contoh:

- Jika user mengetik “ls” maka MyShell akan menampilkan daftar file seperti terminal biasa.
- Jika mengetik “pwd” maka MyShell menampilkan direktori kerja saat ini.

```
} // akhir while
```

```
return 0;
```

```
}
```

Bagian ini menutup loop utama dan fungsi main.

return 0 menandakan program selesai dengan normal.

3. Pengujian dan hasil

```

madha_27@DESKTOP-S58N8I3:~$ nano custom_shell.c
madha_27@DESKTOP-S58N8I3:~$ gcc -o custom_shell custom_shell.c
madha_27@DESKTOP-S58N8I3:~$ ./custom_shell
MyShell> ls
custom_shell  custom_shell.c  multi_process  multi_process.c  process_manager  process_manager.c
MyShell> date
Tue Oct 14 16:37:38 +07 2025
MyShell> pwd
/home/madha_27
MyShell> history
=== 5 Perintah Terakhir ===
1. ls
2. date
3. pwd
4. history
MyShell> exit
Keluar dari MyShell.
madha_27@DESKTOP-S58N8I3:~$ |

```

TUGAS 4

1. Implementasi

- Buat nama file
Nano sysinfo_tool.c

- Ketik codingan dalam file
`#include <stdio.h>`
`#include <stdlib.h>`
`#include <string.h>`
`#include <unistd.h>`

```
void showCPUInfo() {
    FILE *file = fopen("/proc/cpuinfo", "r");
    if (!file) {
        perror("Tidak bisa membuka /proc/cpuinfo");
        return;
    }

    char line[256];
    printf("\n=== INFORMASI CPU ===\n");
    while (fgets(line, sizeof(line), file)) {
        if (strncmp(line, "model name", 10) == 0 ||
            strncmp(line, "cpu cores", 9) == 0 ||
            strncmp(line, "siblings", 8) == 0) {
            printf("%s", line);
        }
    }
    fclose(file);
}

void showMemInfo() {
    FILE *file = fopen("/proc/meminfo", "r");
    if (!file) {
        perror("Tidak bisa membuka /proc/meminfo");
        return;
    }

    char line[256];
    printf("\n=== INFORMASI MEMORI ===\n");
    for (int i = 0; i < 5 && fgets(line, sizeof(line), file); i++) {
        printf("%s", line);
    }
    fclose(file);
}
```

```

void showUptime() {
    FILE *file = fopen("/proc/uptime", "r");
    if (!file) {
        perror("Tidak bisa membuka /proc/uptime");
        return;
    }

    double uptime_seconds;
    fscanf(file, "%lf", &uptime_seconds);
    fclose(file);

    int hours = (int)(uptime_seconds / 3600);
    int minutes = ((int)uptime_seconds % 3600) / 60;
    int seconds = (int)uptime_seconds % 60;

    printf("\n=== WAKTU HIDUP SISTEM ===\n");
    printf("Sudah aktif selama: %d jam, %d menit, %d detik\n",
        hours, minutes, seconds);
}

void showProcCount() {
    system("ls /proc | grep '[0-9]' | wc -l");
}

void menu() {
    printf("\n===== SYSTEM INFO TOOL (NIM GENAP) =====\n");
    printf("1. Tampilkan informasi CPU\n");
    printf("2. Tampilkan informasi Memori\n");
    printf("3. Tampilkan waktu hidup sistem (uptime)\n");
    printf("4. Tampilkan jumlah proses berjalan\n");
    printf("5. Keluar\n");

    printf("=====\n");
}

int main() {
    int pilihan;

    while (1) {
        menu();
        printf("Pilih menu [1-5]: ");
        scanf("%d", &pilihan);
        getchar(); // membersihkan buffer enter

        switch (pilihan) {

```

```

        case 1:
            showCPUInfo();
            break;
        case 2:
            showMemInfo();
            break;
        case 3:
            showUptime();
            break;
        case 4:
            printf("\n=== JUMLAH PROSES BERJALAN ===\n");
            showProcCount();
            break;
        case 5:
            printf("Keluar dari program.\n");
            return 0;
        default:
            printf("Pilihan tidak valid!\n");
    }

    printf("\nTekan Enter untuk melanjutkan...");
    getchar(); // tunggu user tekan Enter
}

return 0;
}

```

- Perintah jalankan
 gcc -o sysinfo_tool sysinfo_tool.c
 ./sysinfo_tool

2. Penjelasan

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

```

Baris ini adalah library yang digunakan:

- `stdio.h` → untuk input/output (`printf`, `scanf`, `fopen`, `fgets`, dll).
- `stdlib.h` → untuk fungsi umum seperti `system()` dan `exit()`.
- `string.h` → untuk manipulasi string seperti `strncmp()`.
- `unistd.h` → untuk fungsi sistem UNIX (digunakan agar kompatibel dengan OS Linux).

```
void showCPUInfo() { ... }
```

Fungsi ini menampilkan informasi CPU dari file sistem Linux `/proc/cpuinfo`.

```
FILE *file = fopen("/proc/cpuinfo", "r");  
Membuka file /proc/cpuinfo dalam mode baca ("r").
```

```
if (!file) {  
    perror("Tidak bisa membuka /proc/cpuinfo");  
    return;  
}
```

Jika file tidak dapat dibuka, tampilkan pesan error dan keluar dari fungsi.

```
char line[256];  
printf("\n=== INFORMASI CPU ===\n");  
while (fgets(line, sizeof(line), file)) {  
    if (strncmp(line, "model name", 10) == 0 ||  
        strncmp(line, "cpu cores", 9) == 0 ||  
        strncmp(line, "siblings", 8) == 0) {  
        printf("%s", line);  
    }  
}  
fclose(file);
```

Bagian ini membaca isi file baris demi baris.

- `fgets()` membaca satu baris teks dari file.
- `strncmp()` digunakan untuk membandingkan awal string.

Jika baris diawali dengan "model name", "cpu cores", atau "siblings", maka baris itu akan ditampilkan.

- `fclose(file)` menutup file setelah selesai digunakan.

```
-----  
void showMemInfo() { ... }
```

Fungsi ini menampilkan informasi memori dari file `/proc/meminfo`.

```
FILE *file = fopen("/proc/meminfo", "r");  
Membuka file memori sistem Linux.
```

```
if (!file) {  
    perror("Tidak bisa membuka /proc/meminfo");  
    return;  
}
```

Jika gagal membuka file, tampilkan pesan error.

```

char line[256];
printf("\n=== INFORMASI MEMORI ===\n");
for (int i = 0; i < 5 && fgets(line, sizeof(line), file); i++) {
    printf("%s", line);
}
fclose(file);

```

Bagian ini hanya membaca 5 baris pertama dari file /proc/meminfo, yang umumnya berisi:

- MemTotal → total RAM
- MemFree → RAM bebas
- MemAvailable → RAM yang masih bisa dipakai
- Buffers dan Cached → memori cache sistem

```

void showUptime() { ... }

```

Fungsi ini menampilkan waktu hidup sistem (lama waktu komputer menyala).

```

FILE *file = fopen("/proc/uptime", "r");

```

Membuka file /proc/uptime.

```

if (!file) {
    perror("Tidak bisa membuka /proc/uptime");
    return;
}

```

Jika gagal, tampilkan pesan error.

```

double uptime_seconds;
fscanf(file, "%lf", &uptime_seconds);
fclose(file);

```

Bagian ini membaca satu angka berupa total waktu aktif sistem (dalam detik).

```

int hours = (int)(uptime_seconds / 3600);
int minutes = ((int)uptime_seconds % 3600) / 60;
int seconds = (int)uptime_seconds % 60;

```

Konversi detik menjadi jam, menit, dan detik agar lebih mudah dibaca.

```

printf("\n=== WAKTU HIDUP SISTEM ===\n");
printf("Sudah aktif selama: %d jam, %d menit, %d detik\n",
    hours, minutes, seconds);

```


Menampilkan hasil waktu aktif sistem dalam format jam, menit, dan detik.

```
-----  
void showProcCount() {  
    system("ls /proc | grep '^[0-9]' | wc -l");  
}
```

Fungsi ini menampilkan jumlah proses yang sedang berjalan di sistem.

- “ls /proc” → menampilkan isi folder /proc.
- “grep '^[0-9]’” → hanya menampilkan nama folder yang diawali angka (karena setiap proses di Linux memiliki ID numerik di /proc).
- “wc -l” → menghitung berapa baris (berapa banyak proses yang ada).

```
-----  
void menu() { ... }
```

Fungsi ini hanya menampilkan tampilan menu utama.

```
printf("\n===== SYSTEM INFO TOOL (NIM GENAP) =====\n");  
printf("1. Tampilkan informasi CPU\n");  
printf("2. Tampilkan informasi Memori\n");  
printf("3. Tampilkan waktu hidup sistem (uptime)\n");  
printf("4. Tampilkan jumlah proses berjalan\n");  
printf("5. Keluar\n");  
printf("===== \n");
```

Menampilkan daftar menu yang bisa dipilih pengguna.

```
-----  
int main() {  
    int pilihan;
```

Variabel pilihan digunakan untuk menampung angka menu yang dipilih user.

```
while (1) {  
    menu();  
    printf("Pilih menu [1-5]: ");  
    scanf("%d", &pilihan);  
    getchar(); // membersihkan buffer enter
```

Loop while(1) membuat program terus berjalan sampai user memilih menu keluar (nomor 5).

Setiap iterasi, program menampilkan menu dan meminta input angka dari user. getchar() digunakan untuk menghapus karakter newline dari input agar tidak mengganggu proses selanjutnya.

```

switch (pilihan) {
    case 1:
        showCPUInfo();
        break;
    case 2:
        showMemInfo();
        break;
    case 3:
        showUptime();
        break;
    case 4:
        printf("\n=== JUMLAH PROSES BERJALAN ===\n");
        showProcCount();
        break;
    case 5:
        printf("Keluar dari program.\n");
        return 0;
    default:
        printf("Pilihan tidak valid!\n");
}

```

Struktur switch digunakan untuk menentukan tindakan berdasarkan input user:

- 1 → panggil fungsi showCPUInfo().
- 2 → panggil fungsi showMemInfo().
- 3 → panggil fungsi showUptime().
- 4 → panggil fungsi showProcCount().
- 5 → keluar dari program.
- Selain itu → tampilkan pesan “Pilihan tidak valid!”.

```

printf("\nTekan Enter untuk melanjutkan...");
getchar();

```

Setelah setiap menu selesai, program akan menunggu user menekan Enter agar user bisa membaca hasilnya terlebih dahulu.

```

}
return 0;
}

```

Akhir fungsi main. return 0 menandakan program berjalan dengan sukses.

□ Alur kerja program:

1. Program menampilkan menu utama.
2. User memilih angka 1–5.

3. Berdasarkan pilihan, program menjalankan fungsi:
 - (1) showCPUInfo → Menampilkan model dan jumlah core CPU.
 - (2) showMemInfo → Menampilkan total dan penggunaan memori.
 - (3) showUptime → Menampilkan lama waktu sistem menyala.
 - (4) showProcCount → Menampilkan jumlah proses aktif.
 - (5) Keluar dari program.
4. Setelah setiap tampilan informasi, user menekan Enter untuk kembali ke menu.
5. Program terus berulang sampai user memilih keluar.

3. Pengujian dan hasil

```
Keluar dari MyShell.
madha_27@DESKTOP-S58N8I3:~$ nano sysinfo_tool.c
madha_27@DESKTOP-S58N8I3:~$ gcc -o sysinfo_tool sysinfo_tool.c
madha_27@DESKTOP-S58N8I3:~$ ./sysinfo_tool

===== SYSTEM INFO TOOL (NIM GENAP) =====
1. Tampilkan informasi CPU
2. Tampilkan informasi Memori
3. Tampilkan waktu hidup sistem (uptime)
4. Tampilkan jumlah proses berjalan
5. Keluar
=====
Pilih menu [1-5]: 1

=== INFORMASI CPU ===
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
siblings        : 8
cpu cores       : 4

Tekan Enter untuk melanjutkan...

Tekan Enter untuk melanjutkan...

===== SYSTEM INFO TOOL (NIM GENAP) =====
1. Tampilkan informasi CPU
2. Tampilkan informasi Memori
3. Tampilkan waktu hidup sistem (uptime)
4. Tampilkan jumlah proses berjalan
5. Keluar
=====
Pilih menu [1-5]: 5
Keluar dari program.
madha_27@DESKTOP-S58N8I3:~$ |
```

KESIMPULAN :

program diatas memudahkan untuk memahami kondisi sistem tanp harus mengetik banyak perintah.